

Vision Transformers: An Attractive Alternative to CNNs

John G. Peters

Talk Overview

Goal: Learn what tools are available for applying pre-trained vision models to specialized problems

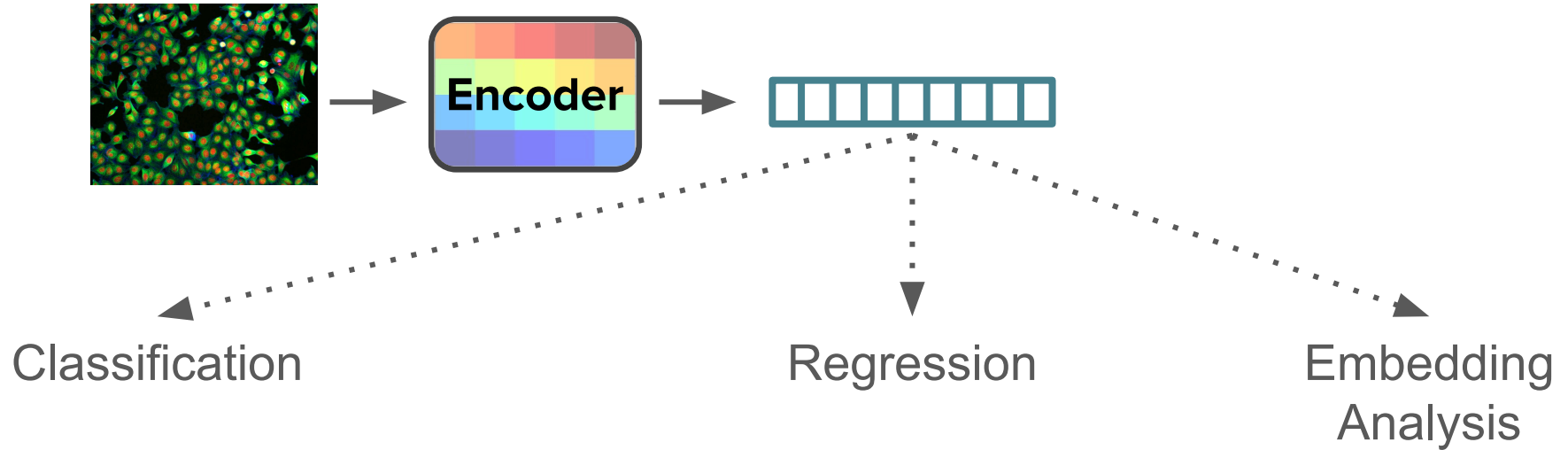
Topics:

- Types of vision models and their applications
- Why and how to utilize pre-trained models
- Step-by step examples of using pre-trained models
- Where expenses add up when adapting them to your problems

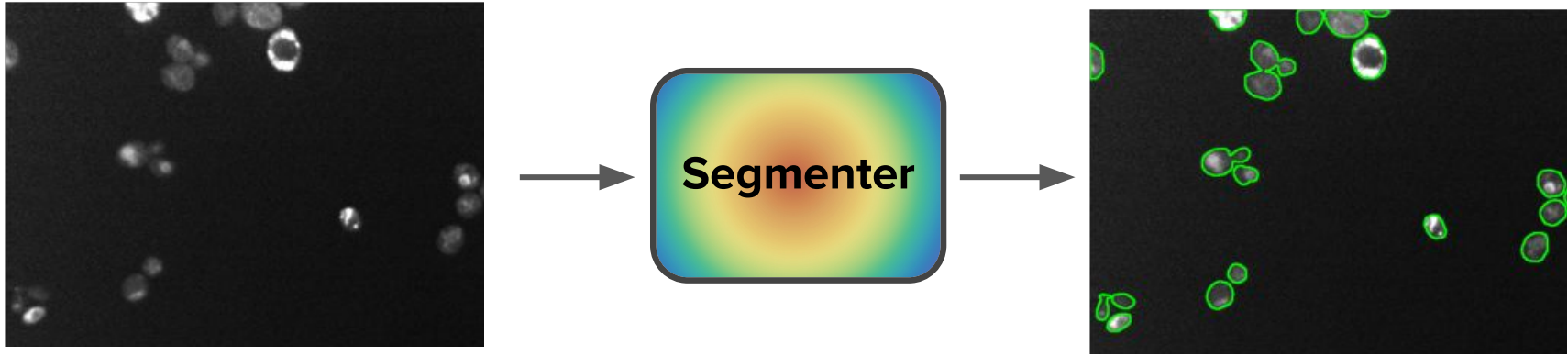
Tip: Example code supplied in QR code at the end!

Vision Modeling Overview

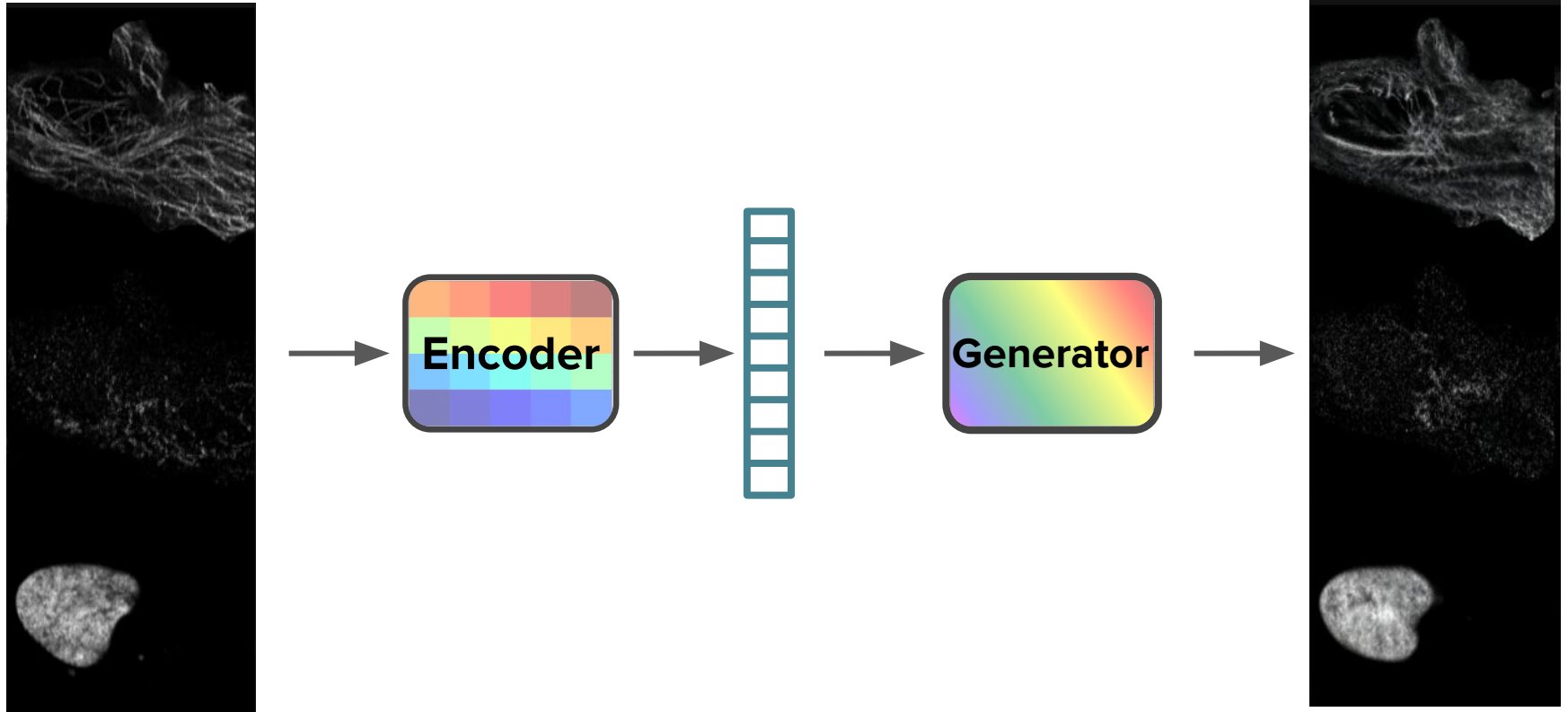
What we use Vision Models for - Representations



What we use Vision Models for - Segmentation



What we use Vision Models for - Generation



Types of Vision Models for different use-cases?

Representation

- CNN
- Vision Transformer
- Convnext
- Mamba

Segmentation

- U-Net
- Segment Anything
- *Yolo*

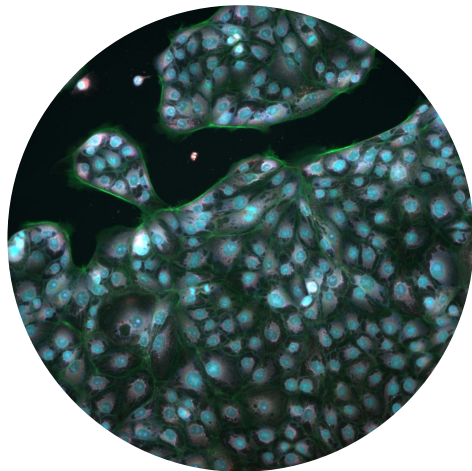
Generation

- Autoencoders
- Diffusion
- Latent Diffusion
- Bitwise
AutoRegression
- Flow Matching
- Normalizing Flow

Why the Vision Transformer?

Why the Vision Transformer?

Reach



Microscopy



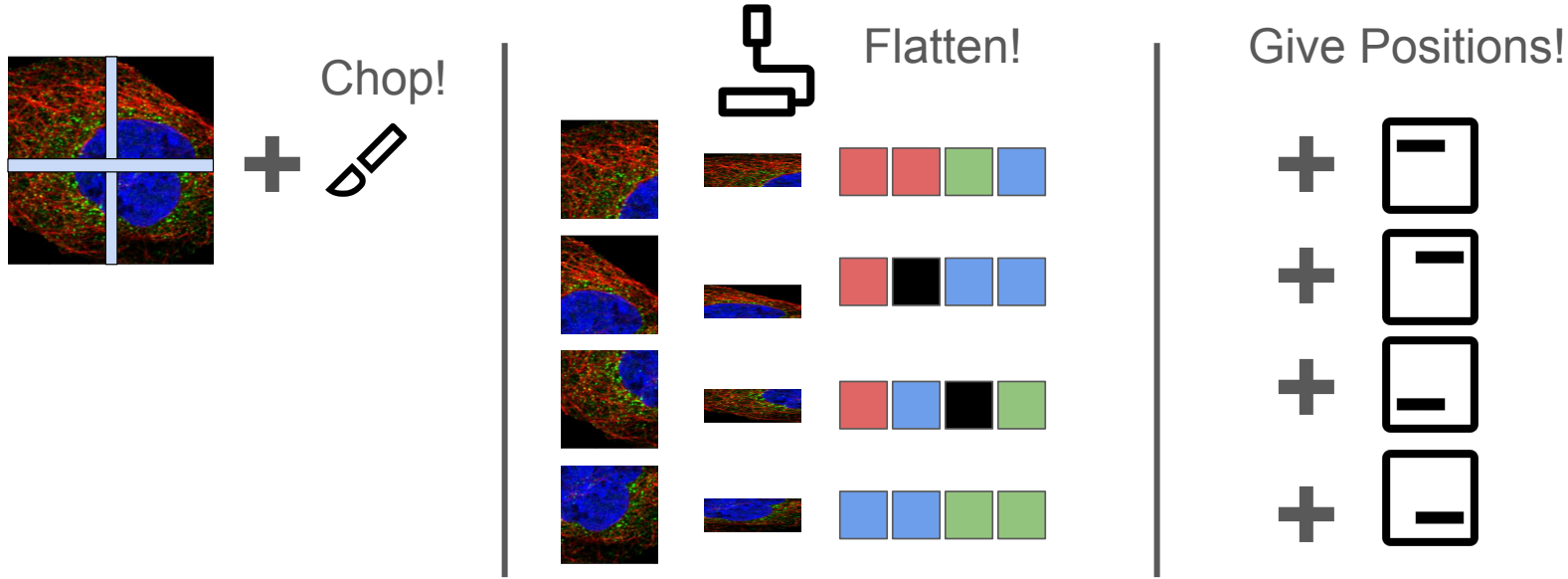
Satellite/Aerial

43°02'44.37"N 89°49'14.01"W



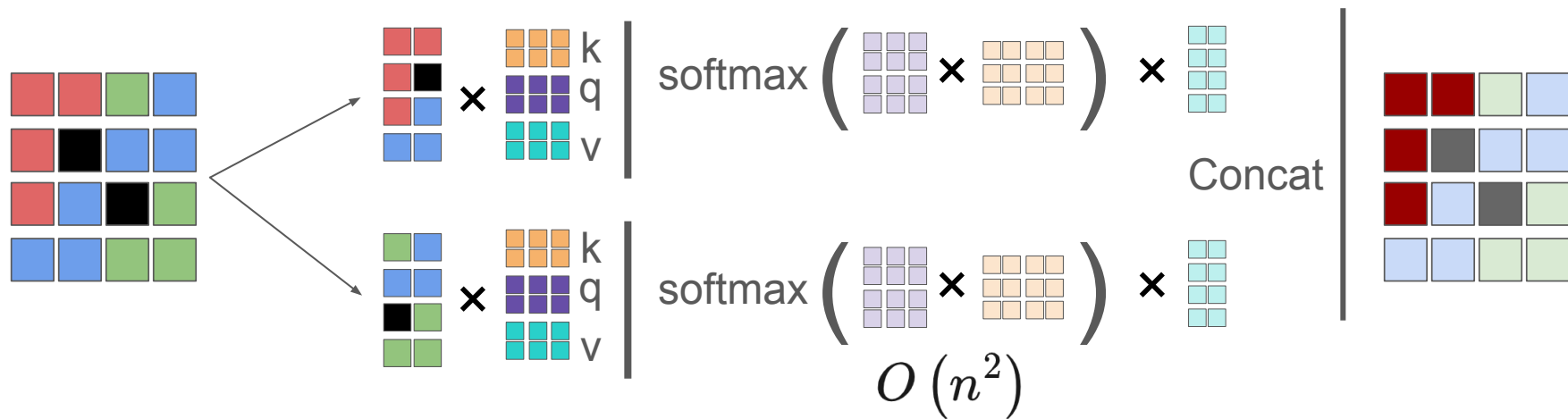
**Zero-Shot
Segmentation**

A little on transformers - patch embedding



Input sequence length defined by resolution of the input image and the chop size

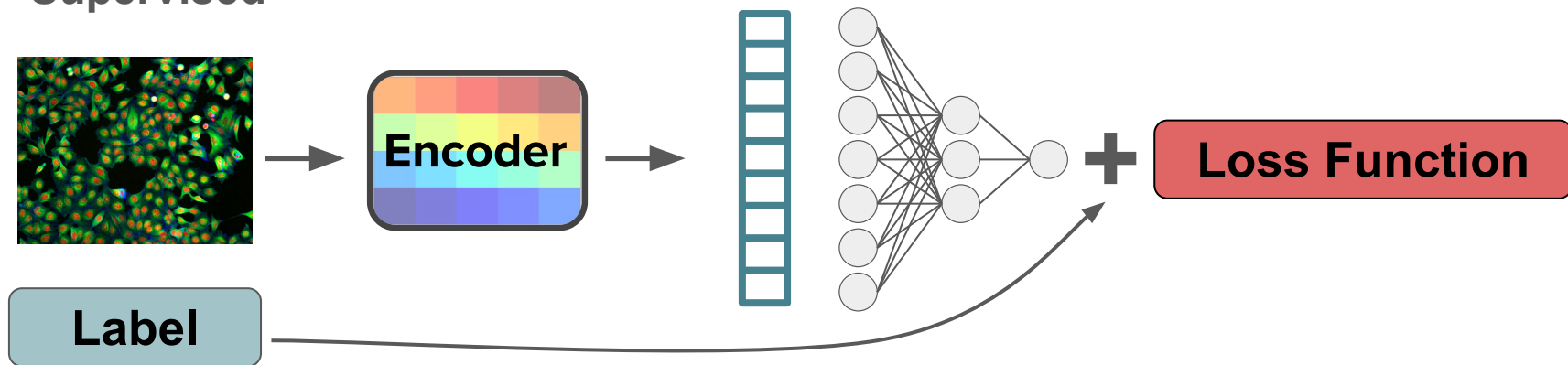
A little on transformers - attention



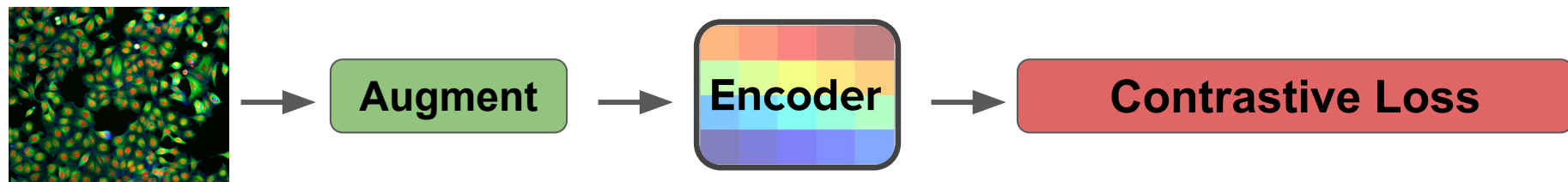
Memory (and often prediction time) scales quadratically with the number of patches (resolution) of an image.

Different training paradigms

Supervised

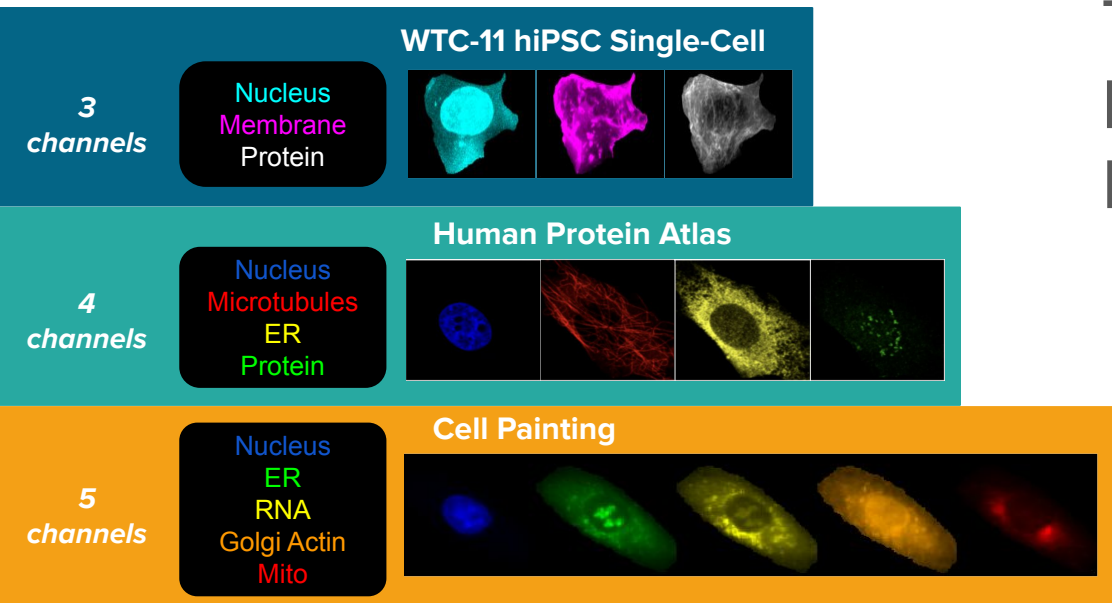


Self-Supervised



Using *Already Spent Compute!*

Benchmark Aside - CHAMMI



Three Datasets
Nine Tasks Total
Higher Score is Better

Fine-tuning

The further training of an already optimized model.

Strategies:

- Supervised fine-tuning
- Self-Supervised fine-tuning

Fine-tuning 101



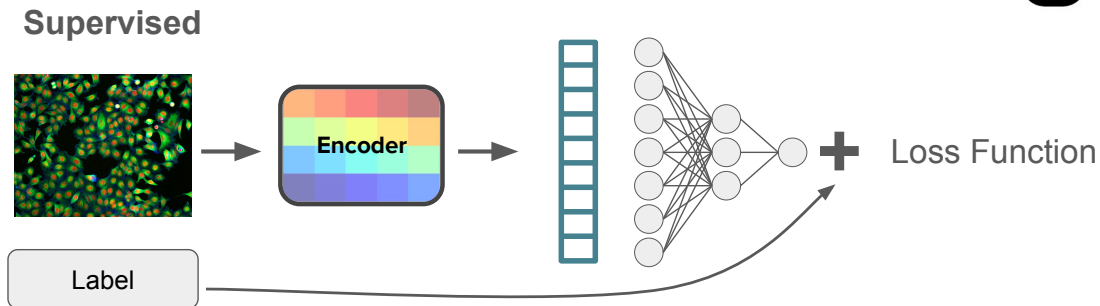
PyTorch Lightning

OR



Accelerate

1. Load pre-trained weights
2. Freeze them to warm-up the prediction head
3. Unfreeze them and finetune for a small number of iterations (<10k)



Fine-tuning example - Finding a Model



Hugging Face

OR



Pytorch Hub

Utilize the 🤗 Hugging Face or 🔥 Pytorch Hub model repositories.

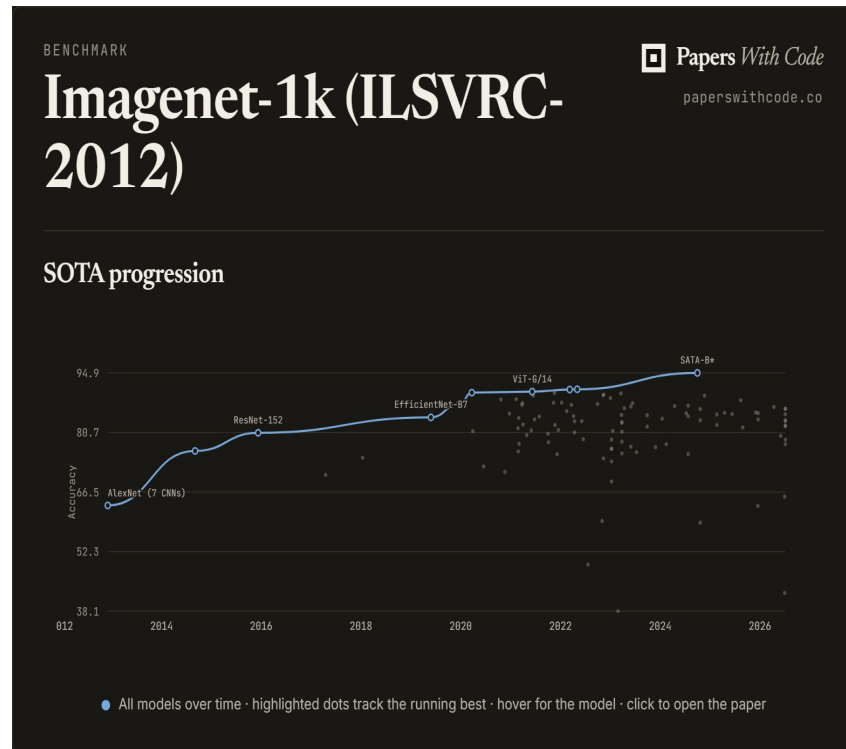
Check on my recommended models at the end of the talk 😊💧

Fine-tuning example -

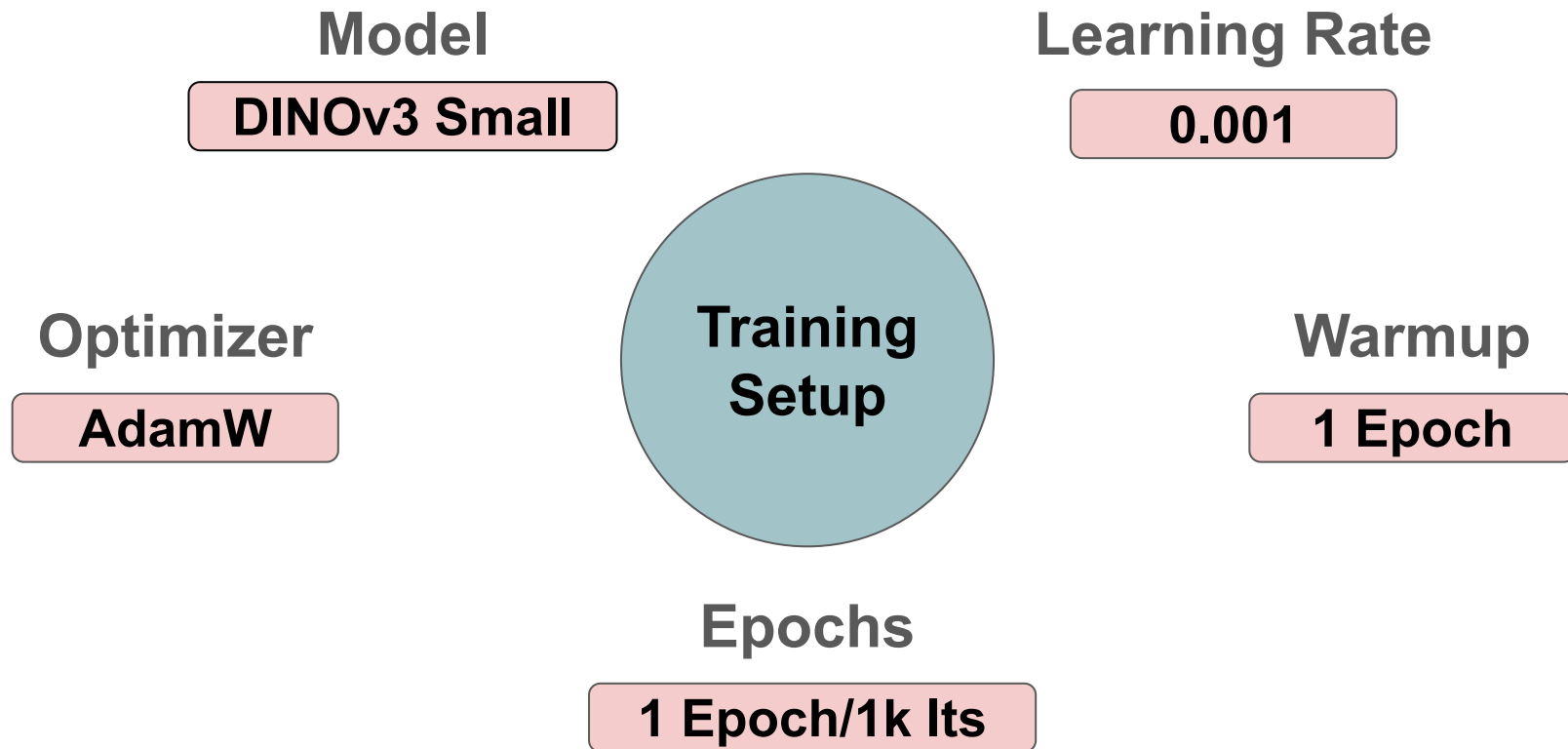


Hugging Face

1. Check out <https://paperswithcode.co/>
2. Click “Browse State-Of-The-Art”
3. Find your task
4. Find the highest on the list by Google/Meta/Other AI Company on

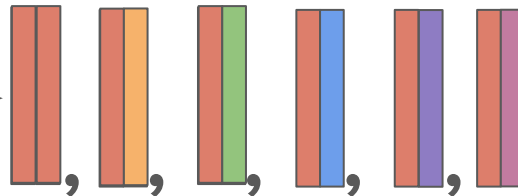


Fine-tuning example - Experimental Setup

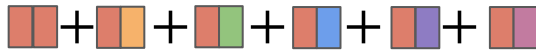
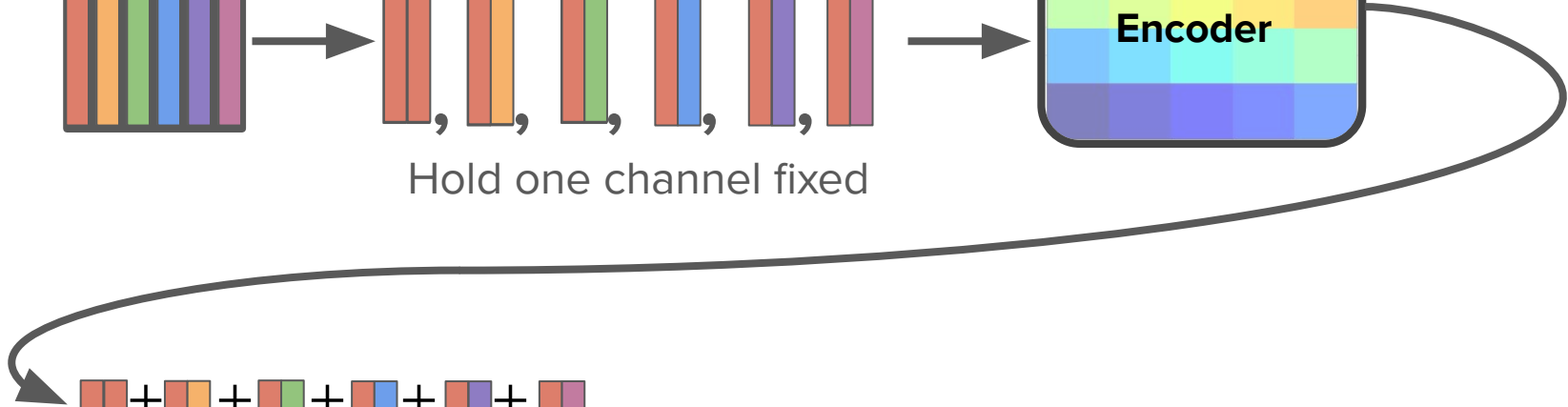
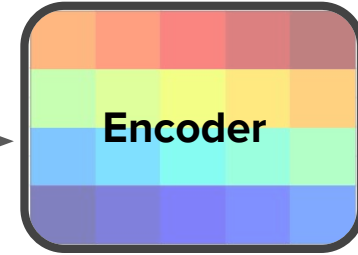


Fine-tuning example - Adapting to new data formats

Multi-Channel
Image



Hold one channel fixed



Concatenate
Morphological profiles

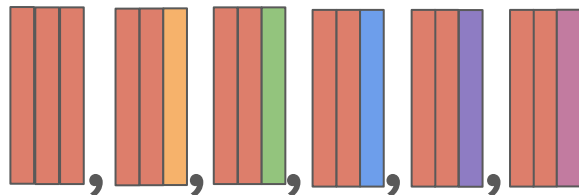
Fine-tuning example - Adapting to new data formats

Expected DINOv3

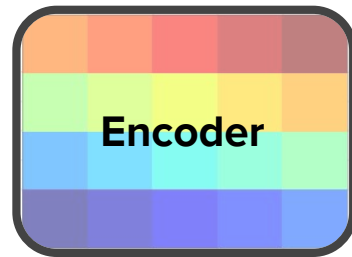
Input



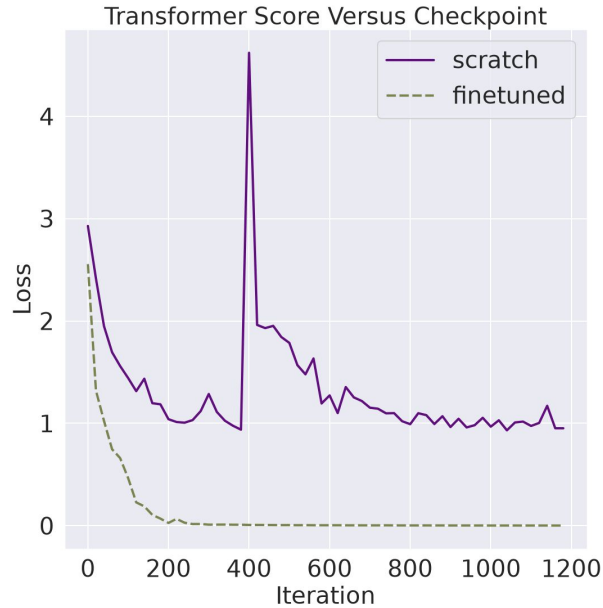
Actual Input Shape



Hold two channels fixed

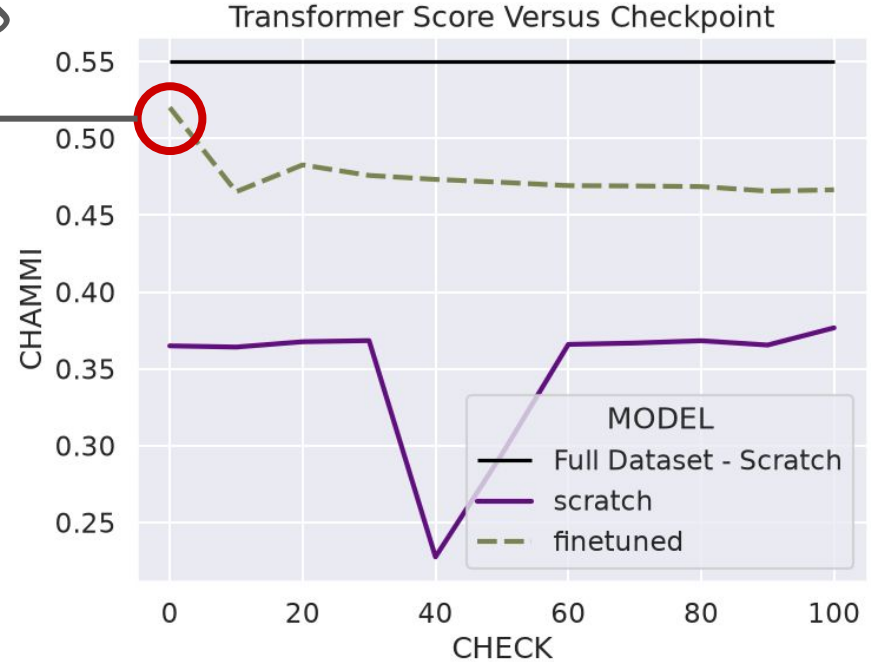


Fine-tuning example - Training time and Performance



6 Minutes, 20 Seconds
~1.2k iterations

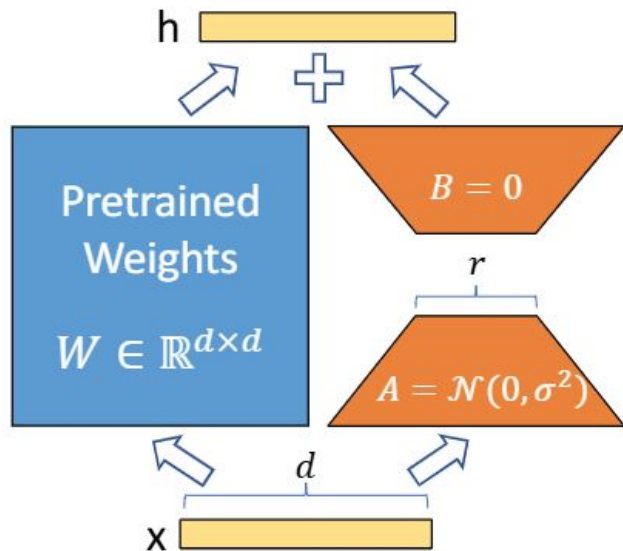
No Training



What about parameter efficient fine-tuning?

Parameter size:

$$r * (A + B) < (d * d) \text{ parameters}$$



Adapts a larger matrix by using two linear layers

A = Gaussian Init

B = Zero Init

The first iteration is a “no change”

LoRA - Finding Parameters

Identify:

- MLP Layers
- Attention Matrices

In DINOv3:

“k,v,q,o”_proj

“up/down”_proj

list(backbone.modules()):

```
(model): DINOv3ViTEncoder(  
  (layer): ModuleList(  
    (0-11): 12 x DINOv3ViTLayer(  
      (norm1): LayerNorm((384,), eps=1e-05, elementwise_affine=True, bias=True)  
      (attention): DINOv3ViTAttention(  
        (k_proj): Linear(in_features=384, out_features=384, bias=False)  
        (v_proj): Linear(in_features=384, out_features=384, bias=True)  
        (q_proj): Linear(in_features=384, out_features=384, bias=True)  
        (o_proj): Linear(in_features=384, out_features=384, bias=True)  
      )  
      (layer_scale1): DINOv3ViTLayerScale()  
      (drop_path): Identity()  
      (norm2): LayerNorm((384,), eps=1e-05, elementwise_affine=True, bias=True)  
      (mlp): DINOv3ViTMLP(  
        (up_proj): Linear(in_features=384, out_features=1536, bias=True)  
        (down_proj): Linear(in_features=1536, out_features=384, bias=True)  
        (act_fn): GELUActivation()  
      )  
      (layer_scale2): DINOv3ViTLayerScale()  
    )  
  )  
)  
(norm): LayerNorm((384,), eps=1e-05, elementwise_affine=True, bias=True)
```


LoRA - Implementation

Utilize 🤗parameter efficient fine-tuning library

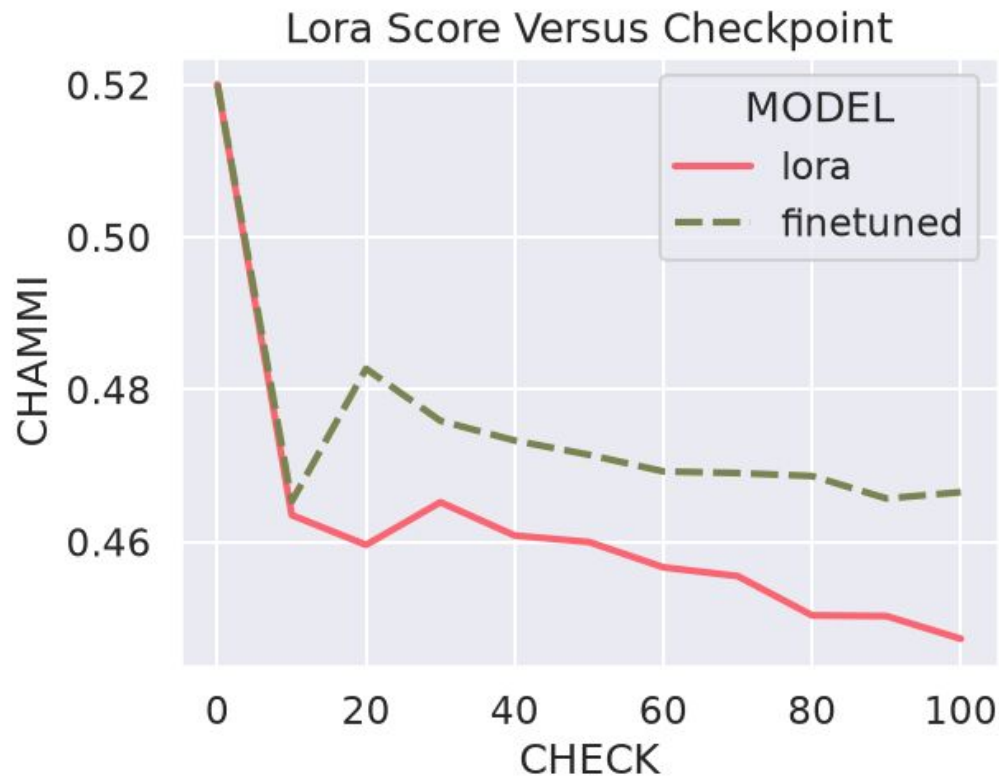
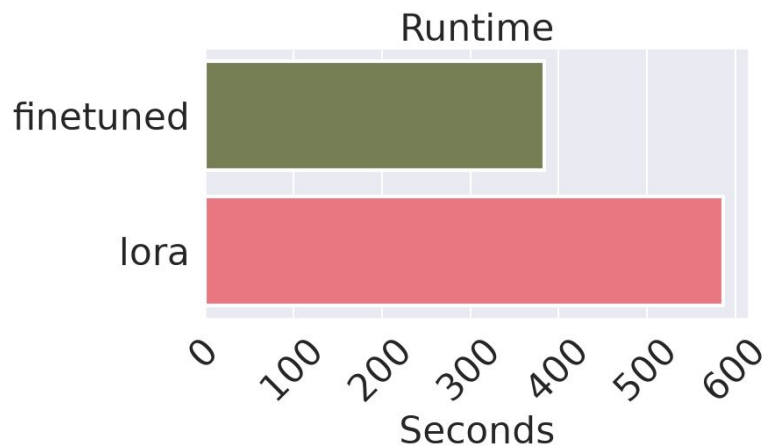
```
from peft import LoraConfig, get_peft_model
def peft_backbone(backbone):
    lora_params = ["k_proj", "v_proj", "q_proj", "o_proj", "up_proj", "down_proj"]
    config = LoraConfig(
        r=256,
        target_modules=lora_params,
    )

    backbone = get_peft_model(backbone, config, autocast_adapter_dtype=False)

    for name, param in backbone.named_parameters():
        if not any(layer in name for layer in lora_params):
            param.requires_grad = True
    return backbone
```

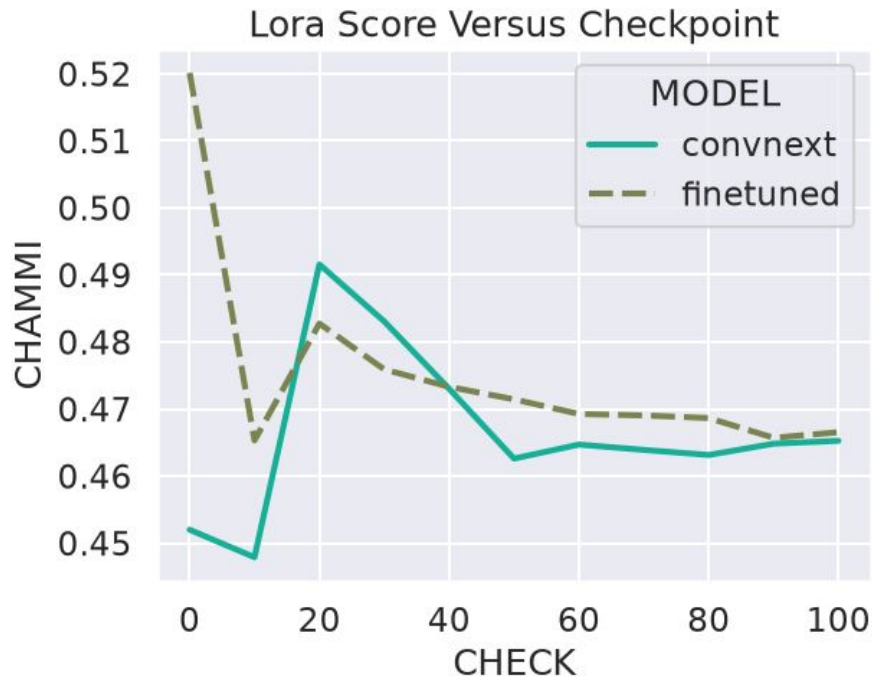
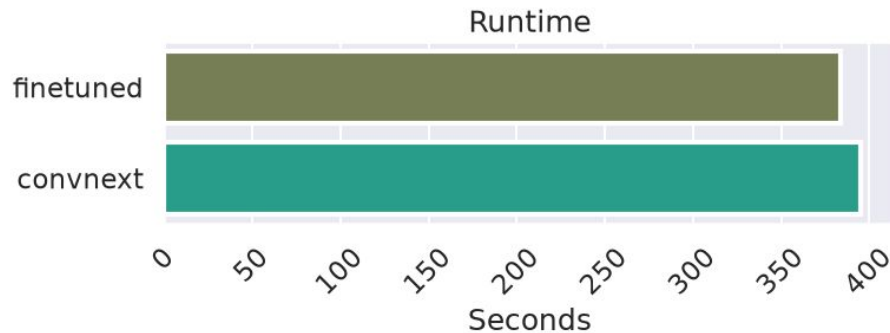
LoRA - Results

- Less Performance
- Longer Runtime



Whoah, but what about CNNs?

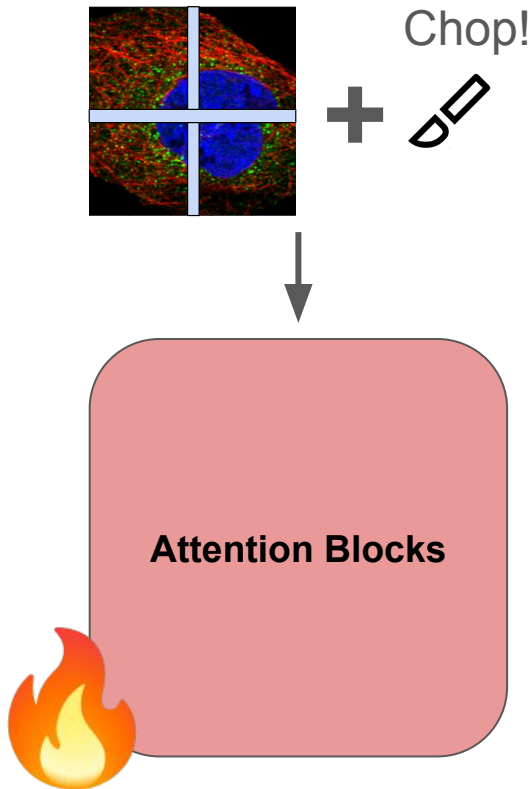
- Performs worse
- Fine-tunes better on limited data
- Runs Slower on 224x224 images



Is fine-tuning strictly worse?

- Self-Supervised
- Entire Training Set
- Newly Learned Patch Encoder
- Initialized Attention Via DINOv3 Weights

DINOv3	Self-Supervised Fine-Tuned	Supervised
52.1	55.2	54.5



Project Costs



Usage Costs

Training

- **Testing Runs**
- **Hyperparameter Tuning**
- **Dataset Creation**
- Training Run
- Data Storage

Inference

- Time on GPU
- Data Storage

Actual Training Costs



One A100 40GB ~ **\$17/hr**

For this talk:

- Convnext fine-tuned (~6 minutes)
- DINOv3 Small LoRA (~6 minutes)
- DINOv3 Small from Scratch (~9 minutes)
- DINOv3 Small fine-tuned (~6 minutes x2)

= ~\$9.5

To train a “Foundation Model”:

- DINOv3 Large on 5TB Images
 - **128 GPUs 8 hrs**

= ~\$10k

Vision Models I recommend for different use-cases:

Representation

- DINOv3 Convnext
- DINOv3 ViT

Segmentation

- Segment Anything

Generation

- Stable Diffusion 3.5
- Infinity

For Cells:

- Morphem
- SubCell

For Cells:

- CellPose-Sam

Talk Outcomes

- Vision Models and their uses
- Applying a pre-trained model
- Fine-tuning Strategies (including LoRA)
- Utilizing LORA
- Why use pre-trained models
- What models to start with

